

PROJECT RETROSPECTIVE

Nova Linux

How a practice ground for Linux commands became an eight-version build

D Shetty | September 2026

8

versions shipped

50+

terminal commands

17

source files at v1.8

103

automated checks, final suite

One goal, sideways: a real-time PC monitoring agent. To build one you must live in the Linux shell, so first I built a Linux to live in - a simulated desktop in the browser where every command is practice, and the worst a bad command can do is wipe localStorage.

1. Why build a fake operating system

The end goal was never a browser toy. It started with a plan to build a real-time PC monitoring agent - a tool that sits on a machine and reports its live state: CPU load, memory, running processes, disk usage. That kind of agent lives on Linux, and it lives in the shell. **ps**, **top**, **df**, **free**, **cat /proc/cpuinfo** - reading their output, parsing their flags and chaining them with pipes is the daily work of writing one.

Practising that on a real machine is uncomfortable: a wrong **rm** eats real files, a dual-boot or VM adds friction, and resetting a mistake means reinstalling. The alternative was to simulate. A Linux desktop that boots in a browser tab in seconds, resets with one click, and cannot break anything it does not own. Nova Linux is that practice ground - every command typed into it is one more rep of muscle memory, deliberately matched to how a real terminal behaves.

Three rules shaped it from the first line of code:

- **One self-contained HTML file** - no build step, no server, runs offline from file:// in any browser.
- **GNU-accurate output** - if a real terminal would not print it that way, Nova Linux does not either: ls -l columns, wc counts, ps tables, and errors like **bash: cd: foo: No such file or directory**.
- **Verified before delivery** - every version ships only after an automated suite drives the OS like a user and passes.

2. One file, a whole desktop (v1.0)

The first version was already the whole idea: a boot sequence, a top bar with a working clock, a dock, draggable windows, desktop icons, and a terminal backed by a virtual file system persisted to localStorage. Roughly 50 commands, pipes, and output redirection, plus four apps - Calendar, Clock, Notes and Settings. It passed 52 automated checks on delivery.

The virtual filesystem became the heart of the project. Directories, files, an in-memory tree serialized to localStorage, a home directory for a user called tux - enough for `cd`, `ls`, `cat`, `grep`, `mkdir`, `rm`, `mv`, `cp` and friends to behave honestly rather than print canned text.

3. Making it feel real (v1.1 to v1.3)

Three rounds of polish turned a demo into something that feels like an OS. v1.1 added a Windows-style taskbar (launcher icons left, labelled buttons for running apps right), resizable windows, a Files app with breadcrumbs, a Help app, and a proper reset/erase split in Settings. v1.2 deepened the shell: **`rmdir`**, **`whereis`**, **`locate`**, **`wget`**, and real symlinks - **`ln -s`** links that show up in `ls`, `tree` and Files. It also introduced Nova AI, an offline assistant that explains Linux commands from a built-in knowledge base of about 65 commands.

v1.3 added the GNOME touches: calendar events with dot markers on days, and a quick-settings panel whose battery actually drains, whose brightness slider actually dims the screen, and whose wifi, bluetooth and airplane-mode pills interact the way they do on a real phone.

4. The desktop shrinks (v1.4 and v1.5)

Below 700 pixels, Nova Linux stops pretending to be a desktop and becomes a phone: full-screen apps, no drag or resize, a navigation bar, a three-column app drawer and a full-width quick-settings sheet. Inputs were set to 16 px to stop mobile browsers zooming on focus. v1.5 fixed the phone app bar overlapping the AI input and Settings erase button - a one-line CSS height fix, found the honest way: by looking at the broken screen.

5. The bug that taught the most (v1.6)

The terminal input was originally an off-screen invisible input with a hand-drawn cursor. On phones, letters appeared in reverse - typing **`mkdir`** produced **`ridkm`** - backspace died, and the caret was invisible. The fake cursor disagreed with the real one because it was a separate thing pretending to be the real one.

The fix was not a patch but a rebuild: a real, visible input sitting in the prompt line, with every key handler reading from it. The bug class was eliminated rather than quieted. It shipped with 18 checks passing and one lesson absorbed: never fake a native control you can have for free.

6. A block cursor that tracks the real caret (v1.7)

One request remained to get the classic look back: a terminal-style block cursor, on laptop and phone alike. The solution keeps the real input as the source of truth but renders it invisible - transparent text, transparent native caret - and underneath draws a mirror of the command line with a blinking block cursor placed at exactly the input's selectionStart.

Because it tracks the selection API rather than guessing, arrow keys move the block, taps place it correctly on mobile, and selecting text renders a highlighted region. Nothing about the native behaviour is fought; the decoration rides on top of it.

7. From single file to source tree (v1.8)

The final round broke the 150 KB single file into a proper project: 17 source files across css/ and js/, one file per app, plain scripts in a documented load order - still no build step, still runs from file://. A build.py script reassembles the single-file version for sharing, and the split was verified lossless by running the same 15-test suite against the folder version. The folder is now the source of truth.

Version	Focus	Headline change
v1.0	Core desktop	~50 commands, virtual FS, window manager, 52 tests
v1.1	Desktop polish	Taskbar, resizable windows, Files and Help apps
v1.2	Shell depth	Real symlinks, locate/whereis/wget, offline Nova AI
v1.3	GNOME feel	Calendar events, quick settings with working battery and brightness
v1.4	Phone mode	Full-screen apps, nav bar, app drawer, quick-settings sheet
v1.5	Phone fixes	App-bar overlap on AI input; terminal padding
v1.6	Input rebuild	Real input fixes reversed typing and dead backspace
v1.7	Block cursor	Caret-tracking terminal cursor with selection highlight
v1.8	Source tree	17-file project split, build.py, lossless reassembly

Version history at a glance

8. Every round shipped tested

Before each delivery the OS was booted in a headless DOM (jsdom) and driven like a user: commands typed into the real input, Enter pressed, windows opened from the shell, quick settings toggled, phone mode simulated by stubbing media queries. The suites assert exact GNU-format output, error strings, symlink display, history recall, pipes and redirects - 103 checks in the surviving suites across v1.0, v1.6, v1.7 and v1.8.

Code changes were applied by patch scripts whose every replacement asserts exactly one match, so an ambiguous edit fails loudly instead of silently corrupting the file. That discipline caught several drafting mistakes before they ever reached a browser. The build environment itself was hostile - sandbox pods restarted and wiped work more than once - so durable backups and restore-on-boot guard scripts became part of the workflow.

9. What is real, and what is pretend

A practice ground only works if it is honest about what it is. Nova Linux draws the line like this:

Subsystem	What is actually happening
Shell parser	Real: quotes, pipes, redirects, append, command history, tab completion
Filesystem	Real semantics: dirs, files, symlinks, permissions display, persistence in localStorage
Window manager	Real: drag, resize, focus, taskbar, phone layouts
Process table	Simulated: ps and top draw a synthetic, self-consistent process list
Network	Simulated: wget and ping fake their output; no network is touched
Nova AI	Offline: a built-in knowledge base, not a live model
sudo	A joke: refused with a wink, like any good practice sandbox

10. The road back to the monitoring agent

The simulation did its job. After eight versions of living inside a shell - parsing ls output, wiring pipes, matching error formats - the commands a monitoring agent needs are now familiar ground rather than reference lookups. The next step is the real thing: a small agent on a real Linux box, reading /proc, sampling top and df, and reporting what it finds. Nova Linux stays as the sandbox to rehearse in, and the parts it taught best - the shell fluency, the testing habit, the honesty about what is real - go with it.

"You do not learn the shell by reading about the shell. You learn it by breaking one that cannot hurt you."